

김민석

프론트엔드 엔지니어 | 주식회사 센디 재직 중

min49590@gmail.com | 010-4601-4954 | GitHub | LinkedIn | 기술 블로그

소개

Next.js, React, TypeScript를 기반으로 물류 서비스의 고객용 웹, 운영 콘솔, 운송관리 웹, 기사 웹을 개발하고 운영해 왔습니다. 복잡한 운영 업무를 웹 UI로 구조화하고, 레거시 시스템을 서비스 중단 없이 점진적으로 현대화하며, 테스트·배포·보안·관측성까지 프론트엔드 시스템 전반을 개선했습니다.

특히 센디 웹의 SEO 개선을 자발적으로 맡아, 페이지 성능 개선 과제를 테크니컬 SEO·온서프 SEO·콘텐츠 운영으로 확장했습니다. 랜딩 페이지 정적화, Core Web Vitals 개선, 구조화 데이터, canonical, 사이트맵, 콘텐츠 시스템을 단계적으로 구축했으며, 개선을 이어간 약 7개월 동안 웹 트래픽이 약 3배로 증가하는 흐름을 확인했습니다.

화면 구현에 그치지 않고 API·세션·상태·캐시·CI·운영 경계를 함께 설계하며, 변경 내용을 재현 가능하고 검증 가능한 형태로 남기는 데 강점이 있습니다. 최근에는 Claude Code와 Codex를 활용한 AI 개발 워크플로우의 인증, QA, PR 검증 방식을 저장소 수준에서 표준화했습니다.

핵심 역량

- **SEO 성장 및 검색·콘텐츠 엔지니어링**: Google Search Console 지표를 바탕으로 렌더링·성능·인덱싱 문제를 개선하고, 정적화·LCP 최적화·구조화 데이터·사이트맵·canonical·콘텐츠 시스템을 연결했습니다. 약 7개월간 개선을 이어가며 웹 트래픽이 약 3배로 증가하는 흐름을 확인했습니다.
- **점진적 아키텍처 현대화**: App Router 전환, 기능 단위 모듈 분리, 세션·상태관리 구조 개선, TypeScript strict 전환을 호환성 경계와 함께 수행했습니다.
- **AI 기반 개발 생산성 개선**: AI 에이전트용 작업 규칙, 브라우저 인증, QA 검증, PR evidence pack, 사람 승인 경계를 저장소 규칙과 자동화로 구현했습니다.
- **복잡한 운영 제품 구현**: 대규모 데이터 테이블, 인라인 편집, 필터·선택·드로어, API 저장, 클라이언트 캐시 갱신을 하나의 사용자 흐름으로 설계했습니다.
- **테스트 및 품질 체계 구축**: Vitest, Testing Library, MSW, Playwright, Biome, dependency-cruiser, knip을 CI와 연결하고 기존 기준선과 신규 회귀를 분리했습니다.
- **운영 안정성 및 보안**: Sentry 기반 오류 수정, 의존성 취약점 차단, 공급망 보호, Docker·ECS 배포 검증과 실패 진단 자동화를 수행했습니다.

경력

주식회사 센디 · 프론트엔드 엔지니어

2023.12.26 - 현재

물류 서비스의 고객용 웹, 운영 콘솔, 운송관리 웹, 기사 웹을 개발하고 운영하고 있습니다.

SEO 성장 및 검색·콘텐츠 시스템

- CTO의 페이지 속도 개선 요청을 계기로 랜딩 페이지의 성능 문제를 조사했으며, 단일 성능 개선 작업을 테크니컬 SEO와 온서프 SEO 전반의 지속적인 개선 과제로 확장했습니다. 공식적인 SEO 직책을 부여받아 시작한 업무는 아니었지만, 실무 오너십을 맡아 지표 점검, 기술 개선, 콘텐츠 기반 구축을 이어갔습니다.
- Lighthouse와 Chrome DevTools의 Performance·Network 분석을 통해 A/B 테스트를 위한 서버 요청 컨텍스트가 정적 생성을 막고, 무거운 폰트·CSS·JavaScript와 비핵심 이미지가 TTFB·LCP 병목을 만드는 구조를 파악했습니다.
- 랜딩 페이지를 `force-static`으로 전환하고, 실험 로직은 hydration 이후 클라이언트에서 적용하도록 분리했습니다. 실험 영역의 크기와 레이아웃을 고정해 정적 HTML의 이점과 A/B 테스트를 함께 유지하면서 CLS 위험을 통제했습니다.

- 사용하지 않는 의존성을 제거하고 무거운 컴포넌트와 서드파티 스크립트를 지연 로딩했습니다. 폰트 웨이트·서브셋, 불필요한 CSS, 이미지 우선순위를 정리해 LCP 병목을 줄였습니다.
- 레거시 랜딩의 `div` 중심 구조를 시맨틱 HTML 로 개선하고, `title`·`description`·`Open Graph`·`canonical` 을 정비했습니다. 페이지 성격에 맞춰 `Organization`, `FAQPage`, `HowTo`, `Article`, `BreadcrumbList` JSON-LD 를 적용하고 리치 결과 인식 여부를 검증했습니다.
- `/blog`, `/guide`, `/review` 등 콘텐츠 섹션별 사이트맵과 `sitemap index` 를 구성하고, 파라미터·유사 URL 의 대표 주소를 `canonical` 로 통제했습니다. Google Search Console 에서 노출수·클릭수·CTR·평균 게재 순위·색인·Core Web Vitals 를 지속적으로 점검했습니다.
- SEO 개선을 이어간 약 7 개월 동안 웹 트래픽이 약 3 배로 증가하는 흐름을 확인했습니다. 구조화 데이터가 검색엔진에 인식되고 실제 검색 결과에 리치 스니펫이 표시되는 것도 확인했습니다.
- 이후 기업 뉴스룸과 공지·업데이트 콘텐츠를 Markdown 기반 정적 콘텐츠 시스템으로 구축했습니다. 목록·상세 페이지, `parser`·`cache`·`query` 모델, `canonical`, 사이트맵, `Open Graph` 와 `JSON-LD` 를 연결하고, 외부 보도를 기업 관점에서 재작성하기 위한 사실 검수 원칙과 작성 가이드를 문서화했습니다.

제품 및 사용자 경험

- 기존 운송관리 화면을 데이터 그리드 중심의 인라인 편집 UX 로 재구축했습니다. 컬럼 크기·순서 저장, 다중·드래그 선택, 필터, 상세·배차 드로어를 제공하고 주소·운송일시·차량·연락처·운임·화물·메모 등의 주요 정보를 셀 단위로 수정할 수 있도록 구현했습니다.
- 단일 셀을 수정할 때 기존 상세 데이터를 바탕으로 안전한 전체 업데이트 `payload` 를 재구성하고, 저장 후 목록 전체를 다시 조회하는 대신 `React Query` 캐시를 패치해 응답성과 데이터 정합성을 함께 고려했습니다.
- 기존 화면은 `legacy` 경로로 유지하고 신규 화면을 별도 경로에 구축해 비교와 롤백이 가능하도록 했습니다. 출시 전 기능은 UI 진입점을 비활성화해 릴리스 시점을 분리했습니다.
- 운영 콘솔에 `Servist` 기반 PWA, `manifest`, `service worker`, `offline fallback` 을 적용하고, 환경별 메타데이터와 `iOS`·`Android` 설치 가이드를 구현했습니다.

아키텍처 및 코드 품질

- 약 199 개의 API 릴레이를 `Next.js App Router Route Handler` 로 이전했습니다. 결제·인증 콜백처럼 리다이렉트와 세션 의존성이 큰 경로는 기존 구조에 남겨 전환 위험을 분리했습니다.
- 코드베이스를 `app`, `core`, `shared`, `features`, `api` 계층으로 재구성하고 기능별 모듈과 `canonical alias` 를 도입했습니다.
- `TypeScript strict` 전환 과정에서 타입 오류를 단계적으로 정리하고, `Biome` 의 타입 안정성 규칙을 CI 게이트로 강화했습니다.
- 사용되지 않던 `Jest` 기반 테스트 환경을 `Vitest`, `Testing Library`, `MSW` 로 교체하고 `Playwright smoke E2E` 와 `GitHub Actions` 의 `typecheck`·`unit test` 검증을 구성했습니다.

AI 개발 워크플로우 및 자동화

- 여러 프론트엔드 저장소에 `Claude Code` 와 `Codex` 가 공통으로 따르는 브라우저 인증, QA 검증, PR 준비 계약을 설계했습니다.
- QA 자격증명은 전용 로컬 프로세스만 읽도록 제한하고, AI 에이전트는 생성된 브라우저 `storage state` 만 사용하도록 분리했습니다. `.env`, 테스트 계정, 쿠키 내용을 직접 읽지 못하도록 금지 규칙과 통합 테스트를 추가했습니다.
- 실제 개발 서버를 사용하는 QA 와 `MSW` 기반 자동 E2E 를 명확히 구분하고, 실행 명령·종료 코드·테스트 수·환경·기준 커밋 SHA 를 `PR evidence pack` 으로 남기도록 자동화했습니다.
- 대표 검증 시점 기준으로 한 저장소에서 단위 테스트 368 건과 E2E 18 건, 다른 저장소에서 단위 테스트 354 건과 E2E 22 건이 통과했습니다. `flaky` 결과와 기존 기준선 오류는 신규 회귀와 구분해 기록했습니다.

운영 안정성, 보안 및 배포

- `Sentry` 에서 확인된 `hydration`, 외부 SDK 로딩, API 실패, `WebView bridge` 문제를 재현 경계별로 분석하고, 404 전환·준비 상태 대기·`null guard`·`optional chaining` 등 원인에 맞는 방어 로직과 회귀 테스트를 추가했습니다.
- `npm audit` 를 `prebuild` 게이트에 연결하고 `Node`, `Nginx`, `Axios`, `PostCSS`, `Sharp` 등의 런타임·의존성 취약점을 단계적으로 패치했습니다.

- 릴리스 경과일 제한, lifecycle script 기본 차단, 전이 의존성 override 를 활용해 공급망 위험을 통제했습니다.
- ECS 배포 workflow 와 Dockerfile 을 정비하고, rollback 검증, 환경별 동시 배포 방지, BuildKit secret, 컨테이너 health check, 실패 단계·원인·로그 요약 알람을 구성했습니다.

주요 프로젝트

센디 웹 SEO 성장 및 테크니컬 SEO 최적화

- 페이지 속도 개선 요청에서 출발해 렌더링, Core Web Vitals, 온서프 SEO, 인덱싱, 모니터링까지 개선 범위를 확장했습니다.
- 랜딩 정적화와 A/B 테스트 경계를 재설계하고, 폰트·CSS·JavaScript·이미지 로딩을 최적화했습니다.
- 시맨틱 HTML, 메타데이터, 구조화 데이터, canonical, 섹션별 사이트맵과 Search Console 운영 체계를 구축했습니다.
- 약 7 개월간 개선을 이어가며 웹 트래픽이 약 3 배로 증가하는 흐름을 확인했습니다.
- [관련 글: 센디 SEO 를 3 배 성장시킨 이야기](#)

운송관리 그리드 기반 인라인 편집 화면 구축

- 복잡한 주문 조회·수정·배차 흐름을 하나의 그리드 경험으로 통합했습니다.
- UI, 상태, API payload, 캐시 갱신, 기존 상세 기능 연결까지 end-to-end 로 구현했습니다.
- 기존 화면 보존과 신규 경로 분리를 통해 점진적 전환과 롤백 가능성을 확보했습니다.

AI QA·브라우저 인증·PR 증거 계약 표준화

- AI 에이전트가 비밀 정보를 직접 읽지 않고 로그인된 브라우저 세션으로 QA 할 수 있는 구조를 설계했습니다.
- 실제 QA 와 mock E2E 를 구분하고, 검증 결과를 커밋 SHA 와 연결해 PR 에서 재현할 수 있도록 했습니다.
- 일부 저장소는 도입 당시 적용이 진행 중이었습니다.

검색엔진·생성형 AI 대응 뉴스로 구축

- 뉴스로 전용 콘텐츠 모델과 정적 페이지를 구축하고 canonical, sitemap, JSON-LD, Open Graph 를 연결했습니다.
- 페이지 개발과 콘텐츠 작성·검수 규칙을 하나의 운영 시스템으로 정리했습니다.

기술 스택

- **Frontend:** Next.js 16, React 19, TypeScript 5, App Router, TanStack Query, Zustand, Tailwind CSS, Storybook
- **SEO & Content:** Google Search Console, Lighthouse, Chrome DevTools, Core Web Vitals, static rendering, sitemap index, canonical, JSON-LD, Open Graph, Markdown content pipeline
- **Testing:** Vitest, Testing Library, MSW, Playwright
- **Quality:** Biome, TypeScript strict, dependency-cruiser, knip, React Doctor, Husky
- **Observability & Experimentation:** Sentry, Datadog, Mixpanel, A/B experimentation SDK
- **Platform:** GitHub Actions, Docker, ECS, Nginx, PWA, Service Worker
- **AI-assisted Engineering:** Claude Code, Codex, agent guardrails, browser QA session isolation, PR evidence automation